

Audit Project Spec

Matteo Franzil, Valentino Armani, Luis Augusto Dias Knob, Domenico Siracusa

July 10, 2024

This document contains the project specification.

Last revision: May 21, 2024

Audit Project

The scope of this project is to devise, implement, and evaluate a system that parses in real-time cloud audit logs, extracts relevant information, and generates a graph that captures the interactions between the events in the logs.

Such graph captures the relationships between the events and the entities involved, the temporal dimension of the events, and the context in which the events occurred, essentially providing an abstracted and high-level view of the actions that are performed in the cloud environment.

With this system, we aim to provide a tool that can be used for real-time anomaly detection, as well as for post-mortem investigations, by both allowing security experts to query the graph and by providing a visualization of the graph that can be used to understand the relationships between the events.

Motivation

- Cloud Auditing is a powerful security feature that as it is does not realise its full potential. Indeed, some tools present in the market perform audit cruching and already provide some intelligence over the incoming information, but no one performs actual cross-log correlation between temporally distinct events. Furthermore, the such tools tend to be very verbose and generate a lot of false positives due to the inherent background noise of the cloud environment.
- Other academic works focus on doing data forensics and post-mortem investigation over system audit logs. However, these works usually work with a temporally limited snapshot of the logs, and do not provide a real-time analysis of the logs. Most importantly, their focus is on the human interpretability of the logs, as security experts are the ones that usually perform the analysis and need to quickly identify the relevant part of the logs that contains, e.g., an intrusion. In our case, our focus is to facilitate the machine analysis of the logs, so that we can perform real-time anomaly detection without putting unnecessary burden on the model to perform the interpretation of the log itself.
- GUI and querying are secondary: the graph should be quick to be generated and to parse, but in the end a machine will query it.
- To the best of our knowledge no one explored the capabilities of audit logging for doing real time anomaly detection over cloud environments.

Objectives

1. Parse audit logs, extracting relevant information and discarding redundant and non-informative data
2. Store the parsed data in a structured format that captures:
 - the relationships between the events and the entities involved
 - the temporal dimension of the events
 - the context in which the events occurred
3. Evaluate the performance of the system in terms of:
 - the correctness of the generated structure
 - we should investigate the loss of information and the choices that must be made between preserving data and discarding it
 - the effectiveness of the structured data
 - the information that can be extracted
 - the ease-of-use of the data structure and the preservation of causality relationships
 - the efficiency of the parsing process, e.g.
 - the spatial and temporal complexity in traversing the data
 - the time required to parse a log file
 - the memory footprint of the system
 - its feasibility as a real-time system (not necessarily)

Timeline and milestones

(V) = Valentino, (M) = Matteo

DONE Weeks 1-2 (May 7 - May 20)

- (M+V) Literature review
 - (audit) log parsing and analysis
 - graph-based data structures
 - data provenance
- (M+V) Initial investigation of K8s audit logs
 - Set up infrastructure to collect logs
 - Identify priority events to parse
 - * Assess possible problematic areas
- Results
 - ☒ A literature review (shared) document
 - ☒ A shared cluster with K8s running to collect logs from
 - ☒ Initial rough idea of events to focus on

DONE Weeks 3-6 (May 21 - June 17)

- (M+V) Designing the system
 - Choosing which data structure to use
 - Outlining a definitive priority list of events to parse
 - * This also includes which Service Accounts to focus on first
- (V) Initial implementation of parser for logs
 - Parse JSON logs
 - Extract relevant information
 - Build a data structure to store the parsed data
 - * Optimize for fast access and traversal
- (M) Work on understanding the relationships
 - Understand: when I do X, what is generated? Who is involved?
 - Which events are related to each other? Which can be discarded?
- Results
 - ☒ The design of the system
 - ☒ An ordered list of *milestones* to reach, w.r.t. the capabilities of the system, along with the difficulties in doing so

DONE Weeks 7-8 (June 18 - July 1)

- (M+V) Studying and implementing the model for category 1 logs
- (V) Connect the parser to the data structure, creating a pipeline that can parse logs, tag and store the information, and on-demand generate the graph
- Results
 - ☒ A working prototype of the system, able to parse logs of category 1, that generates a graph that summarizes the interactions

TODO Weeks 8-10 (July 2 - July 16)

- (M) Designing category 2 scenarios, creating logs and adapting the model to the new data
- (M) Work on hyperparameter tuning, model improvement
- (V) Initial development of a log sorting and visualizing tool that takes a labeled log and divides it into action
- Results
 - ☒ A model that efficiently parses category 2 logs
 - ☒ Results of the model and hyperparameter study
 - ☒ First prototype of the visualizer

WAIT Weeks 11-12 (July 17 - July 30)

- (V) Focus on: improving the visualizer
 - The visualizer should be able to take a log and divide it into actions, and then show the actions in a way that is easy to understand
- (M) Implement and improve for category 3 and 4 logs
- (M) Further studies to improve the model.
 - Focus must be ease to train, and the ability to generalize to new logs
- Results
 - ☒ A model that efficiently parses category 3 and 4 logs
 - ☒ Further graphs and reports on the model capabilities, accuracy, and efficiency
 - ☒ An improved version of the visualizer

WAIT Weeks 12-16 (July 30 - August 26)

- Refining queries
 - Could be the right moment to start making *user stories* to understand how the system could be used and what features users would like to see
 - Strong focus on the temporal dimension
 - * Assume an incident happened at time X, what happened in timespan $[X - 1h, X + 1h]$?
 - * Is it possible to "replay" the events that led to the incident?
- Work towards feature-completeness
 - The system should be able to parse all the events we set out to parse
 - Each event should be thoroughly tested, the choices made strongly justified and documented
- Did we meet the list of milestones we set up?

Yes Perfect, we can move on to the next phase

No Take a time machine and go back to the previous weeks to fix the issues

WAIT Weeks 17-18 (August 27 - September 9)

- (M+V) Starting the publication draft
 - The document should generalize the work done, making it suitable for not just Kubernetes logs but for any kind of audit log (e.g. AWS, GCP)
 - Still to be decided which type of publication we are aiming for
- (M+V) Final assessment of the system
 - The system should be able to parse logs in a reasonable amount of time, and the data structure should be efficient in storing the data
 - The system should be able to answer queries in a reasonable amount of time
 - The query system should be user-friendly, and the results should be easily interpretable
 - The system could have some kind of visualization to help users understand the data

- If something went wrong, this is the correct moment to apply fixes and improvements to the system.
- Results
 - A first WIP of the publication, still in the early stages of writing
 - A final assessment of the system
 - A visualization of the data structure
 - A final report on the project

WAIT Weeks 19-25 (September 10 - October 28)

- Writing the publication
- Finalizing the system
- Closure work
- Results
 - A publication
 - A final report on the project
 - A final version of the system